IPSE-DISCO

Integrated Platform for SEmiconductors DISCOvery

V1.0.1

User's manual

Written and maintained by:
Dr. Gabriele Saleh, Italian Institute of Technology (IIT)
contacts: gabriele.saleh@iit.it ; gabrielesaleh@outlook.com

Table of Contents

Table of Contents 2
General code description 3
Installation / Getting Started (mostly Unix) 3
IPSE_DB module 5
How to run 5
Input keywords: general format rules 5
Input keywords list 5
Output files 7
Input/output example 8
IPSE_ML module 9
General description 9
Module capabilities 9
Advanced features 10
Dataset 10
How to run 10
Input keywords description 10
IPSE_CH module 15
How to run 15
Input Keywords description 15
General options: 15
ML options 16
External File options 17
MongoDB options: 17
Acknowledgements 18
Appendix A. Semiconductor-specific feature representations 19
Appendix B. Density estimation procedure. 20
Appendix C. Setting up MongoDB database (local or AWS cloud). 20

General code description

IPSE-DISCO[1] is a python-based computational platform aimed at predicting stable materials with desired properties. While some features are designed for semiconductors, the platform can be applied to any type of material. Its overarching goal is to bring together all the computational tools that can be useful for the purpose of materials design: big data, machine learning, and atomistic simulations. This is accomplished through the use of a number of modules[2], which run independently from each other, but can be chained together (i.e. the output of one module can be given in input to another, *e.g.* IPSE_DB to IPSE_ML). This aims at making the platform also useful for more specialized tasks such as fetching materials from databases. The modules which have been already implemented in ver. 1.0.1 are listed below.

- IPSE_DB: interface to materials databases. This module connects, *via* the internet, to materials databases (AFLOW, NOMAD, Materials Project, OQMD) and downloads data (including crystal structures) for those materials that fulfill user-defined criteria.
- IPSE_ML: machine learning. This module implements machine learning (ML) algorithms to predict materials properties (ver. 1.0.1 mostly focuses on formation energies). ML algorithms can be trained on a set of materials, and the resulting ML model can be saved on external file and adopted to make predictions on new materials. The training set of materials can be produced by the IPSE_DB module, but external training sets provided by the user can also be adopted.
- IPSE_CH: convex hull. Calculates the convex hull of a chemical system, given a set of compounds and formation energies. These can be provided manually, taken from materials databases (IPSE_DB), or calculated through ML (IPSE_ML). X-ray diffraction patterns can also be systematically simulated.

Some of these IPSE-DISCO modules implement some features specific for semiconductors, such as the band gap filter in IPSE_DB or the anion-cation feature representation (see Appendix A) in IPSE_ML. It is however possible to disregard these features and use this code for any class of materials.

Installation / Getting Started (mostly Linux)

The IPSE-DISCO is directly run through python (see "how to run" sections of the various modules). However, it relies on a number of libraries, which have to be set up before running the code. There are two ways to do that: pip and conda, as explained below. As of ver. 1.0.1, there are two sets of libraries, one for the DB module and another one for the ML and CH modules (these two modules share several objects).

The first step is in any case to clone the repository (alternatively, one can download it 'manually' from the GitHub website):

```
git clone https://github.com/ilgabri/IPSE_DISCO.git
cd IPSE_DISCO
```

Then, one can decide whether to use pip or anaconda/miniconda (miniconda recommended).

To install it through pip:

1. (optional) Create a virtual environment:
   ```
   python -m venv venv_ipse
   source venv_ipse/bin/activate
   ```
2. Install dependencies:
   ```
   pip install -r <requirements_file>
   ```
   Where <requirements_file> indicate either the file *requirements_DB.txt* or the file *requirements_ML_CH.txt*.
3. Run the code as described in this manual (see the "how to run" section relative to each *IPSE* module)

[1] Latin for *I learn it myself*

[2] Note that here the word "module" does not refer to a Python module, it is a general term to indicate an independent part of a program.

Through (mini)conda (see here for installing miniconda):

1. Create the Conda environment with desired name:

   conda env create -f <environment_name.yml> [--name <desired_environment_name>]

   Where <environment_name.yml> indicates either the file environment_DB.yml or the file environment_ML_CH.yml ; <desired_environment_name> indicates the desired name for the environment, in case the user wishes to change it with respect to the IPSE default.
2. Activate the environment:

   conda activate <environment_name>
3. Run the code as described in this manual (see the section relative to each *IPSE* module)

Contact G. Saleh (gabrielesaleh@outlook.com) if you experience issues with the above library setting procedure.

IPSE_DB module

This module can be used to search materials in materials databases. The user can define the criteria for the search in terms of composition, band gap, stability *etc.* (see "Input keywords list"). At the end of the run, the program returns a list of materials fulfilling the user's criteria along with their (available) properties, and, if requested by the user, the crystal structures of those materials (see "Output files" section). As of ver. 1.0.1, the interface to the following databases is implemented: Materials Project[3], AFLOW[4], NOMAD[5], OQMD[6].

How to run

Simply run the main file through python:

```
python IPSE_DB.py <input_file> [<output_file>]
```

Where <input_file> is a plain text file where the user may insert some keywords from those listed below, in any order (free format). Optionally, the user can specify the name of the main output file (see Output files section), otherwise it will be named *IpseDisco.out* .

Input keywords: general format rules

- ✓ Input keywords are case insensitive (except chemical element names and few other cases explicitly stated in the "input keywords list" section)
- ✓ Hashtags and exclamation marks define comments (meaning that the text following # or ! until the end of the line is not processed)

Input keywords list

(actual keywords shown in italic in this section)

- *database*: Select the materials database in which the search is performed. Allowed options: *aflow*, *nomad, mp, oqmd*. Default: *aflow*.
- *getstruct*: if this keyword is present, all the structures of the compounds matching the user-defined parameters are downloaded in the common format of VASP[7] code (*i.e.* POSCAR/CONTCAR[8]). The files are named <formula>_<database><compoundID>.vasp (e.g. Bi5Cu4S10_oqmd2124193.vasp). No argument required (flag keyword).
- *directgap* (not allowed for OQMD database[9]): if this keyword is present, only materials with a direct band gap are fetched from the selected database. No argument required (flag keyword).
- *indirectgap* (not allowed for OQMD database[9]): if this keyword is present, only materials with an indirect band gap are fetched from the selected database. No argument required (flag keyword).

---

[3] Jain, A., Ong, S. P., Hautier, G., Chen, W., Richards, W. D., Dacek, S., ... & Persson, K. A. (2013). Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL materials*, *1*(1).

[4] Curtarolo, S., Setyawan, W., Hart, G. L., Jahnatek, M., Chepulskii, R. V., Taylor, R. H., ... & Morgan, D. (2012). AFLOW: An automatic framework for high-throughput materials discovery. *Computational Materials Science*, *58*, 218-226.

[5] Draxl, C., & Scheffler, M. (2018). NOMAD: The FAIR concept for big data-driven materials science. *Mrs Bulletin*, *43*(9), 676-682.

[6] Kirklin, S., Saal, J. E., Meredig, B., Thompson, A., Doak, J. W., Aykol, M., ... & Wolverton, C. (2015). The Open Quantum Materials Database (OQMD): assessing the accuracy of DFT formation energies. *npj Computational Materials*, *1*(1), 1-15.

[7] https://www.vasp.at/

[8] https://www.vasp.at/wiki/index.php/POSCAR

[9] As of February 2025, OQMD does not contain information on the direct/indirect character of the band gap. This keyword is ignored when *database* is *oqmd* .

- *gapwidth* : range of allowed band gap width (in eV) for materials to be fetched from the selected database. Allowed options: any two real numbers (minimum and maximum allowed band gap value, in eV). Default: none (any band gap value is allowed).
- *numspecies* : number of atom types contained in the chemical formula of the materials to be fetched from the selected database. If not provided, the number of atom types is determined automatically from the atom types keywords (*ion_A*, *ion_B*, *ion_C* , see below) as the minimum number of atom types that allow to contain all elements requested (that is, 3 plus one for each additional atom types defined by colon in the atom types keywords). A value of -1 can be used to allow any number of atom types. Allowed options: any integer. Default: see above.
- *ion_A* , *ion_B*, and *ion_C* (aliases: *ion_A* , *ion_B*, and *ion_hal* or *ion_cat*, *ion_chalc*, and *ion_hal*): sets of atom types (*i.e.* element names) allowed in the materials fetched from the selected materials database. Atom types are separated by spaces or commas: they are connected by the boolean operator "or". For each of these keywords, it is possible to provide more than atom type lists by separating them with a colon with spaces before and after it (" : "): in this case, the colon-separated lists are joined by the boolean operator "and". This can be useful to search double perovskites (e.g. "ion_B Fe,Co : Os" for a double perovskite in which the B cations can be either Fe and Os or Co and Os) or mixed-anion perovskites (e.g. ion_hal F,Cl : Br,I for perovskite containing any of the following couple of halogen: F+Br, F+I, Cl+Br, Cl+I). Note that also the three lists provided by ion_A,ion_B, and ion_C are joined by "and". Finally, the keyword "any" can be used instead of atoms lists to to allow any atom type for a given ion. Allowed options: element names (case sensitive!) separated by commas or spaces (or colon to increase the number of atom types, see above), or just *any*. Default: *any*.
- *check_perovskites* : if this keyword is present, for each material found, the code checks its crystal structure and determines whether it has a (3D) perovskite structure. The algorithm implemented in pymatgen ("coordination_sequence" function) is adopted, which is based on the relative position of the B cations in ABX_3 (e.g. Pb in $CsPbBr_3$). What atom types are considered B ions is defined by the keywords *ion_B* (see above). No argument required (flag keyword).
- *pagesize*, *maxresults*, and *npages* (ignored for searches in the Materials Project[10]): number of results (*i.e.* materials) per page, maximum number of results, maximum number of pages[11], respectively. Clearly, these three keywords are related to one another. If *npages* is not provided, it is calculated by dividing *maxresults* by *pagesize* (if the latter is not provided by the user, it is assigned a value of 50). If both *npages* and *maxresults* are provided, *maxresults* is ignored. In fact, the code internally only adopts *npages* and *pagesize* as variables. Allowed options: integer numbers (one per keyword). Defaults as follows. Aflow: none, all results fetched at once. Nomad: *pagesize* 200 , *npages* 50. OQMD[12]: *pagesize* 50, *npages* 100 (larger pagesize in oqmd often creates error in the server response).
- *E_above_hull* (only available in Materials Project and OQMD databases). Distance from the convex hull (see *e.g.* [13] for convex hull definition), provided as a range of values. Units are eV/atom. Allowed options: any two real numbers. Default: None (any distance from the convex hull is allowed).
- *icsd_only* (not available in NOMAD): if this keyword is present, the query will only retrieve materials which are also present in the Inorganic Structural Database (icsd). No argument required (flag keyword).

---

[10] As of February 2025, Materials Project does not allow pagination info to be provided through their API. It is only possible to use the *num_chunks* and *chunk_size* options, which are however not enough to perform the search page by page. To do that, one would need to use the options *_page*, *_per_page*, *_skip*, *_limit* , which are not available in the python API of Materials project; one would need to manually build a get request, as discussed in this blog post.

[11] Here "page" refers to a chunk of data fetched from the database at once. These paginations option only regulate the access to databases, they don't influence the final result of the IPSE_DB code.

[12] Note on OQMD: as far as the author could see, OQMD does not allow to filter by range of values. So, post-fetching filters are applied if range of values are selected (*e.g.* for the band gap). Thus, in these cases, the number of actual results may be lower than those selected by the pagination variables.

[13] Bartel, C. J. (2022). Review of computational approaches to predict the thermodynamic stability of inorganic solids. *Journal of Materials Science*, *57*(23), 10475-10498.

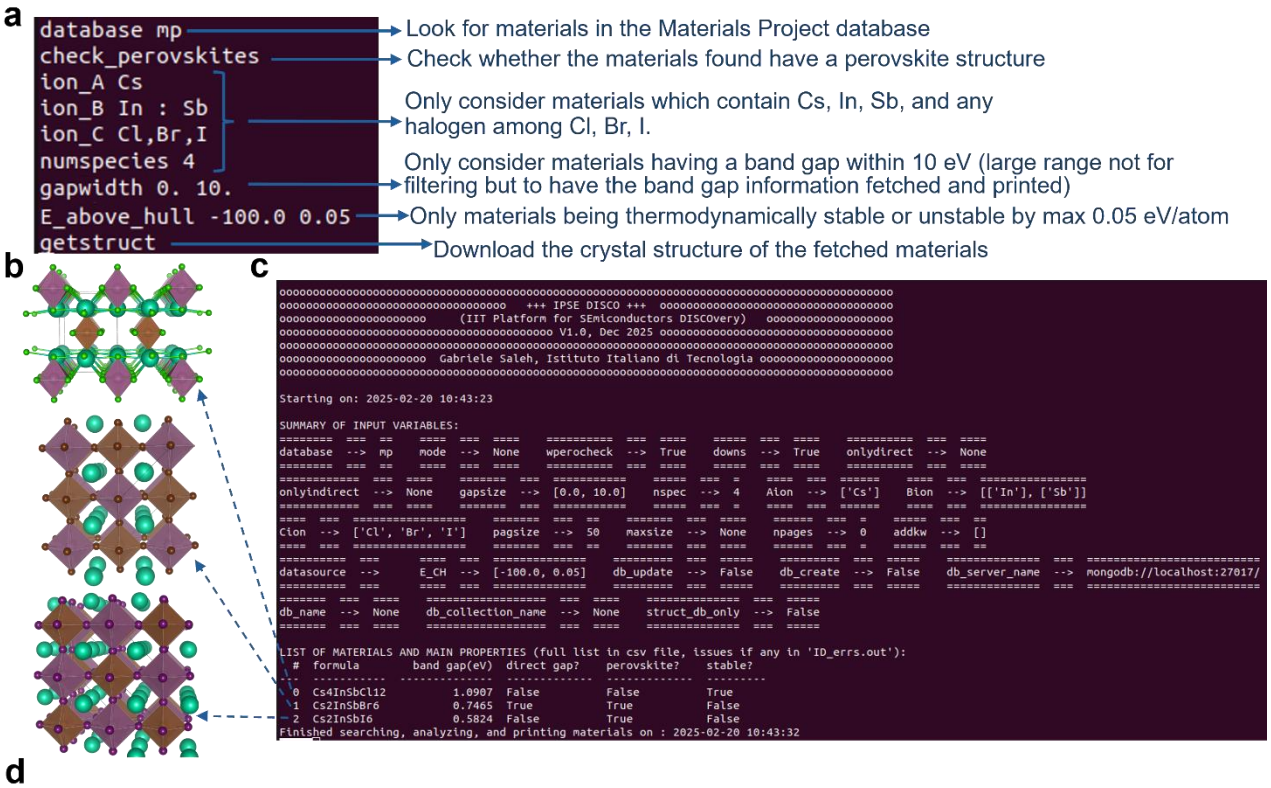
- *db_create*: create a MongoDB in which the results fetched from materials databases are stored. Allowed option: string. See Appendix C. Default: none (no MongoDB will be created).
- *db_update*: update an existing MongoDB database with the results fetched from materials databases. Allowed option: string. Default: none (No MongoDB will be updated).
- *struct_db_only*: store the crystal structures in the MongoDB database without saving them as single POSCAR files (see *getstruct* keyword). No argument required (flag keyword).
- *db_server_name*: address of the MongoDB server on which the results will be stored when *db_create* or *db_update* options are present. See Appendix C. Allowed options: string. Default: none.
- *db_name*: name of the MongoDB database to be created/updated. Allowed option: any string. Default: none (an error message will appear if *db_create* or *db_update* is present but *db_name* is not provided).
- *db_collection_name*: name of the collection (within the MongoDB database) where the results fetched from materials databases are stored. Note: a further collection will be inserted into the database, named "<name_of_collection_selected_by_the_user>_queries", where the query parameters are stored, so that the user can always trace back the query used to create a given collection. Allowed options: string. Default: none.
- *aflow_addkw* (AFLOW only): additional properties to be retrieved for materials. Allowed options: strings separated by space or comma; those strings must be among the keywords listed at: https://aflow.org/documentation/ . Default: none.
- *key_pw* (Materials Project only, mandatory): the key provided by the Materials Project upon registration. It is mandatory to provide it when making searches in the Materials Project database. Allowed options: string. Default: none.

Output files

The program produces three output files:

- Main output file. If the output name is not provided when launching the program, this file is named *IpseDisco.out*. This plain text file contains a summary of the input parameters, running time, and a list of the fetched materials with their properties.
- csv file. If the name of the output file was provided by the user, this file has the same name with the csv extension added, otherwise it is named *IpseDisco.csv*. It contains all the requested details (when available) about all materials fetched during the search in materials databases.
- [optional] CONTCAR files. Crystal structure files in VASP format. These files are generated if requested in the input file (see Input keywords list section)

Input/output example



	A	B	C	D	E	F	G	H	I	J	K	L	M
	#	formula	band gap(eV)	direct gap?	perovskite?	stable?	E_form(eV)	E convex hull (eV)	software	DFT functional	ID (within database)	E_total(eV/atom)	E_MP_uncorrected
	0	Cs4InSbCl12	1.091	False	False	True	-1.8347721408493	0	vasp	MP_MixScheme	mp-1225983	-3.24224985555555	-3.24224985555555
	1	Cs2InSbBr6	0.746	True	True	False	-1.4723004655977	0.02083163	vasp	MP_MixScheme	mp-1113094	-3.004481399	-3.004481399
	2	Cs2InSbI6	0.582	False	True	False	-1.1026062425977	0.03365505375	vasp	MP_MixScheme	mp-1113283	-2.660026824	-2.660026824

Figure 1. An example of database search (ver. 1.0.1): Pb-free double perovskite containing Cs, In, Sb, and one halogen among Cl, Br, and I. a) input file with keywords explanation; b) crystal structures of the materials fulfilling the criteria defined in panel *a*; c) text output file (IpseDisco.out). Note how the two structures labelled as perovskites actually feature the recognizable vertex-sharing octahedra of 3D perovskites in panel *b*; d) output csv file (IpseDisco.csv), which can be opened in Microsoft Excel and similar software, containing detailed information about the materials found in the database.

IPSE_ML module

This module implements machine learning (ML) algorithms to predict the properties of materials. As of ver. 1.0.1, the properties prediction is based only on the chemical composition of the material (*i.e.* no structural information is required). Through this module, the user can train a ML model on a dataset of materials, typically obtained through the IPSE_DB module (see below), save the trained model and use it to make predictions on new materials, including exploring the convex hull of a chemical system (IPSE_CH module).

General description

The software builds a (regression) ML model that connects the stoichiometry of a material[14] to one of its properties, such as the formation energy or the band gap. Thus, in order to train the model, a dataset containing a list of material formulas and the corresponding target property is needed (see also dataset section). This is used to perform the so-called *training* procedure, through which the ML model 'learns' the stoichiometry-property relationship. The trained model can then be saved on a file and later adopted to predict the properties of new materials given their stoichiometry. This *prediction* procedure can be applied to materials for which the target property is already known, in order to appraise the accuracy of the ML model on predicting the properties of new materials. In this case, the prediction procedure is termed *validation*[15].
Given a material dataset, the prediction accuracy of a ML model is determined by two factors: the feature representation and the ML algorithm. Below, these two concepts are briefly described in the context of the IPSE_ML module.

- Feature representation indicates, in the present code, the algorithm to create a set of features out of a chemical formula (ver. 1.0.1). Figuratively, it is the recipe to convert the chemical formula into a language that can be effectively adopted by a ML algorithm. IPSE_ML allows the user to choose and test a set of different feature representation strategies, see Input keywords (and Appendix A).
- The ML algorithm is a set of mathematical functions and procedures adopted to describe the target property in terms of the material features. Examples of ML algorithms are k-nearest neighbors and random forest. As of ver. 1.0.1, the scikit-learn library is adopted, thus the available ML algorithms are those implemented in scikit-learn. However, the code is structured in such a way that different libraries can be adopted without the need to restructure the code.

Module capabilities

This section lists all the capabilities of the IPSE_ML module. See keywords description and examples for the details on how each capability is implemented.

- Dataset creation. Datasets for ML training and validation can be read from files or created from MongoDB collections, see Dataset section). This latter option allows to implement specific filtering criteria that cannot be implemented when requesting data from materials databases; moreover, the filters applied to MongoDB are much faster than filtering in python.
- ML model training. A ML model can be trained on the created datasets to predict a material property given its brute formula. The typical k-fold cross-validation is adopted to estimate the accuracy of the ML models and select the best model.
- Validation/prediction. The previously trained ML models can be saved on a file (pkl format) and be adopted to predict the properties of new compounds not included in the training dataset.

[14] Note: for a given formula, there are many possible crystal structures, and each of them has different properties. This would make the composition-property relationship ill-defined. To overcome this issue, composition-based ML models implicitly assume that the property of interest is obtained/predicted on the lowest-energy crystal structure of each composition.
[15] Note that in the ML literature sometimes this set of data is called 'test set'. We use the terms *training set* and *test set* for the cross-validation procedure (see module capabilities section), and the term *validation* for the held-out set.

Advanced features

- *Features analysis*. The features for the ML model can be analyzed to estimate the correlation among them and their importance in determining the target property.
- *Features combination*. Features can also be combined to create new features. This procedure serves to account for non-linear relationships between the features and the target.
- *Importance-based feature reduction*. The ML model can be re-trained using only the most important features, so as to avoid overfitting issues.

Dataset

The data to train the ML models can be read either from files (csv or free text formats) or directly from MongoDB databases. This capability creates a direct interface with the IPSE_DB module. Thus, the data fetched from materials databases through the IPSE_DB module can be saved locally (or on *cloud* through MongoDB) and be used directly for ML training. Moreover, when reading data from MongoDB, it is possible to filter them by various criteria related to their composition and/or their properties (see MongoDB keywords within the input keywords). This feature allows to apply specific filtering criteria that are not available when querying the databases; moreover, filtering directly in the MongoDB is much faster than post-processing the data in python.

How to run

As of ver. 1.0.1, the input options are set directly in python format in the file *input_IPSE_ML.py* (default values are set for most keywords, see Input keywords description Section). The IPSE ML module is thus simply run by launching python with the main file (add "> *log_file &*" to run the code in the background and print general information on the *log_file*) :
python IPSE_ML_main.py

Note that, unlike in the IPSE_DB module, where the user is only required to build an input file and run the code, this module is not meant to be used as a black box, especially when building the ML model. This is because ML training requires some understanding of the ML implementation within the code, which is described in this document. Nonetheless, the user can still build basic ML models (and use them to make predictions) by minimally filling up the input file and running the code with the 'default' settings.

Input keywords description

General keywords

(keyword names and values in italic. Next to each keyword, the type of input is reported in parentheses).

- *skipDB* (boolean). If *True*, training data are read from a file (csv or free text format), not from mongoDB. Default: *False*.
- *skipML* (boolean). If *True*, the program only fetches data from the MongoDB and possibly writes them to a file, without doing any ML fitting/prediction. Default: *False*.
- *excluded elements* (list of strings or *None*). Compounds containing these elements are excluded from the database search and the properties of these elements are not fetched from the atomic properties database. If *None*, all elements that are not cations or anions (see below) are excluded. Example: *excluded elements = ["Pb", "Cd", "Hg"]*. Default: *None*.
- *cations* (list of strings). Elements to be considered as cations. This has effect both on the mongoDB search (only compounds containing at least one cation and one anion are included in the dataset) and on the ML training/prediction (See Appendix A). Default: *["Li","Na","K","Rb","Cs", "Be","Mg","Ca","Sr","Ba","Sc","Ti","V" ,"Cr","Mn","Fe","Co","Ni","Cu","Zn","Y"*

,"Zr","Nb","Mo","Tc","Ru","Rh","Pd","Ag","Cd","La","Hf","Ta","W","Re","Os","Ir","Pt","Au","Hg","Al","Ga","In","Tl","Sn","Pb","Sb","Bi"]

- *anions* (list of strings). Elements to be considered as anions. This has effect both on the MongoDB (only compounds containing at least one cation and one anion are included in the dataset) and on the ML training/prediction (See Appendix A). Default: *["F","Cl","Br","I","O","S","Se","Te","As"]*
- *fraction_validation* (float or *None*). Fraction of data to be used as validation set (or saved as validation set if *skipML=True*). *None* not to do any validation. Default: *0.05*.
- *file_name_data_fitting* (string). File name of the csv file where training/test data are saved (if *skipDB=False*) or can be read from (if *skipDB=True*). ".csv" extension is automatically added. Default: *train_test_set*.
- *file_name_data_validation* (string). File name of the csv file where validation data are saved (if *skipDB=False*) or can be read from (if *skipDB=True*). ".csv" extension is automatically added. Default: *validation_set*.

MongoDB keywords.

- *db_server_name* (string). Name (address) of the MongoDB server. Default: *"mongodb://localhost:27017/"* (which is the address typically assigned to the MongoDB installed to the user's own computer).
- *db_name* (string). Name of the MongoDB database from which data are read. Default: *None*.
- *db_collection_name* (string). Name of the collection within the *db_name* database from which data are read. Default: *None*.
- *n_elements_mongo* (list of integers or None). Only compounds containing these number of elements are fetched. If *None*, no filter based on the number of elements is applied. Example, to fetch only binary and ternary compounds: *[2,3]*. Default: *None*.
- *density_deviation_filter* (float or *None*). Since databases can contain crystal structures made by isolated fragments, which are most likely not the lowest-energy structure of the corresponding composition, this option allows to empirically estimate the density based on the composition (see appendix B) and discard compounds whose density deviates more than *density_deviation_filter* times from the estimated density. If the density is not directly available from the dataset, IPSE_ML tries to calculate it from the structure. If the structure is not available either, the code prints an error message. Since the density estimation is empirical and semi-quantitative, it is advised to use a large threshold such as 1.0. Use *None* not to apply this filter. Example, to discard compounds whose density differ by more than 80% from the estimated value, use: *density_deviation_filter=0.8* . Default: *None*.
- *various_filters* (list of strings). Options to filter out some compounds based on their properties. As of ver. 1.0.1, the allowed options are: *"noEform"* – exclude compounds whose formation energy was not reported, *"warnings"* – exclude compounds with a warning associated (labelled during the search in the IPSE_DB module), *"errors"* – exclude compounds with error associated (labelled during the search in the IPSE_DB module). Default: *["noEform"]*.
- *bandgap_filter* (list with two floats or *None*). Range of band gaps outside which compounds are filtered out. *None* retains all materials regardless of their band gap. Default: *None*.
- *mongo_fields_to_consider* (list of strings). What materials properties are retrieved from MongoDB. If composition-based ML is adopted, the minimum list to be adopted is "formula" and the property to be predicted (e.g. *"Eform"*). Additionally, "density" is needed if *density_deviation_filter* is adopted. Default: *['formula','Eform','density']*.
- *save_mongo_to_csv* (boolean). If True, data fetched from mongoDB are saved csv files, whose names are set by *file_name_data_fitting* and *file_name_data_validation* (see General keywords). Default: *True*.

Basic ML keywords.

- *styles_ML_features* (list of strings). Feature representations to be adopted (see Appendix A and *options_ML_styles*). Allowed options as of ver. 1.0.1: *"matminer"* – to use feature representation from the matminer library (https://hackingmaterials.lbl.gov/matminer/featurizer_summary.html), "ID1" – statistical distribution of selected elemental properties, "ID2" – minimum formal charges, "ID3" – distribution of elements within the periodic table, "ID4" – statistical distribution of atomic orbital energies and radii (these styles are described in details in Appendix A). Default: *["matminer","ID2","ID3","ID4"]*.
- *options_ML_styles* (dictionary where keys are *styles_ML_features* strings and values are lists of keywords). For some of the feature representation styles defined above, various options can or must be selected. Below a list of possible key-values as of ver. 1.0.1:
    - *"ID1"*: select which elemental properties are considered. As of ver. 1.0.1, allowed options are (the default is to use all of them)[16]: *'atomic_radius_rahm','atomic_volume','covalent_radius_bragg','covalent_radius_cordero','dipole_polarizability','e_affinity','en_allen','electronegativity_martynov-batsanov','en_pauling','electronegativity_sanderson','en_ghosh','ionization_energy','mendeleev_number','metallic_radius','vdw_radius_alvarez','vdw_radius_mm3'*. See their definition at https://mendeleev.readthedocs.io/en/stable/data.html.
    - *"ID2"*: no options needed, as of ver. 1.0.1.
    - *"ID3"*: *'periods', 'groups', 'entropy'*. See their definition in in Appendix A. Default: [*'periods', 'groups', 'entropy']*
    - *"ID4"*: no options needed, as of ver. 1.0.1.
    - "matminer": any feature representation from matminer, see: https://hackingmaterials.lbl.gov/matminer/featurizer_summary.html.
- *styles_atomic_data* (list of integers). What elemental properties to fetch from the database. It is not recommended to change the default, that is: *styles_atomic_data=[1,2,3,4]* .
- *fitting* (boolean). Whether to perform the ML fitting on training/test data. Default: *True*.
- *validation* (boolean). Whether to adopt the ML model to predict the quantity of interest on some data, typically not included in the training/test set (validation or prediction). Default: *True*.
- *do_features_analysis* (boolean). If *True*, feature analysis is performed and printed on the file *features_analysis.out*. As of ver. 1.0.1, by default the analysis consists in the feature importance (permutation importance and, if tree-based algorithms are used, the corresponding feature importance) and in the cross-correlation among features. Other methods for feature importance evaluation can be switched on (see *shapley_feature_importance* and *dropcolumn_feature_importance*). Default: *False*.
- *n_test_train_splits* (integer). The number of batches in which the training set is split to perform the k-fold cross validation. Default: *5*.
- *ML_algorithms* (list of strings). Which ML algorithms are adopted in the training/test/validation procedure. A fair set of hyperparameters is also tested. The allowed options are as follows. *"ETs_depth_scan"*: extra trees with various max_depth values, *"ETs_alpha_pruning_scan"*: extra trees with various ccp_alpha values (pruning), *"ETs_def"*: extra trees with default parameters, *"nuSVR_scan"*: nu support vector regression with various nu/C/kernel, *"GBR_scan"*: gradient boosting regressor with various #estimators/learning rate/subsample/max features, *"HGB_scan"*: Histogram-based Gradient Boosting Regression Tree with various learning rate/max iterations/L2 regularization, *"RF_scan"*: random forest scan with various #estimators/learning rate/max features/bootstrapping, *"kNN_scan"*: k-nearest neighbors regressor with various #neighbors/weights/p, *"XGB_scan"*: XGBoostRegressor with various #estimators/learning rate/subsample/colsample_bytree, *"GPR_scan"*: gaussian process regressor with various kernels.

[16] For ionization energies, the first two ionization energies are adopted. This can be changed by changing the *nIE* variable in the *data_for_ML.py* file.

Once a scan is performed, the user can perform a new training/test/validation with a specific algorithm by adopting the keyword *"PICK_<algorithm name>"*[17]. Default: ["ETs_def"].

Advanced ML keywords.

- *try_features_combination* (boolean). If *True*, the program tries to augment features by adding combinations of them according to the combination operators listed in *features_combinator_operators*. First a training with an L-regularized linear model is performed adopting the algorithm defined in *regularized_model_featcombo*. Then, those feature combinations that have a non-zero coefficient upon regularization are retained and added to the original set of features. The usual training/test/validation procedure is performed using the augmented features set defined by the initial features plus those combinations with non-zero coefficient. Note: to perform a training/test/validation adopting only the most important features, this keyword has to be set to *True* and *features_combinator_operators* has to be set to *None*. Default: *False*.
- *regularized_model_featcombo* (string). This keyword selects the L-regularized model that is used to select the relevant (*i.e.* non-zero upon regularization) feature combinations, as described in the keyword *try_features_combination*. Allowed options: *"elasticnet"*, *"lasso"*, *"elasticnetCV"*, *"lassoCV"*, *"SVR"* (which stands for support vector regression; not exactly a regularized model). Default: *"elasticnet"*.
- *features_combinator_operators* (list of strings or *None*). List of operations adopted to combine features (see *try_features_combination* keyword). Allowed values: *"polynomial"*, *"+"*, *"-"*, *"*"*, *"/"*, *"^2"*, *"^3"*, *"^-1"*, *"exp"*. *"polynomial"* adopts the namesake procedure from scikit-learn with degree 2 (i.e. degree 2 polynomial combination of all pairs of features). *"+"*, *"-"*, *"*"*, and *"/"* perform the corresponding operation on the all the possible pairs of (initial) features. "^2", "^3", "^-1", and "exp" are feature combinations in the proper sense; they apply the operation on each feature. Note that more than one operation can be used at the same time. If *features_combinator_operators=None* , no feature combination is performed. Note: to perform a training/test/validation adopting only the most important feature but without any feature combination, one must set *features_combinator_operators=None* and *try_features_combination=True.* Default: None.
- *threshold_important_features* (integer or float). When *try_features_combination=True*, this keyword sets the threshold of importance above which features are adopted to perform the training/test (and validation if *validate_with_impo_feats=True*) procedure. If an integer N is given instead of a float, rather than retaining the features above an importance threshold, the N most important features are retained. With this set of important features, the training/test is performed and the results are written in the file "fitting_extended.log". If *None*, the training/test/validation with the most important features is not performed. Default: *None*.
- *shapley_feature_importance* (boolean). Whether to perform Shapley feature importance analysis[18]. Default: *True*.
- *dropcolumn_feature_importance* (boolean). Whether to perform feature importance analysis through the drop-column method (for each feature, a 5-fold cross validation is performed without that feature, and the resulting accuracy drop on the test set is computed). Default: *False*.
- *file_important_feats* (string). If *do_features_analysis=True , try_features_combination=True, features_combinator_operators=None*, and *threshold_important_features* is neither *0* nor *None*, the label of the most important features according to *threshold_important_features* are saved on this file. If, instead, *do_features_analysis=False* and *threshold_important_features=None*, the code performs additional fitting and/or validation using only the features whose labels are present in this file. Default: *None* (i.e. important features are neither read from nor written on a file).

[17] To adopt a specific ML algorithm from sklearn (or othe library), the user can modify directly the ML instances in the file Instances_MLalgos.py (instructions on how to do so are written within the file).
[18] https://shap.readthedocs.io/en/latest/

- *validate_with_impo_feats* (boolean). If *True*, the most important features (see *threshold_important_features*) are also adopted for the validation (i.e. to predict target properties on data left out from the training/test set). Default: *False*.
- *ncpus_features* (integer). Number of cpus over which the feature calculation is parallelized. This procedure does not scale very well with the number of cpus unless the training set is large ($>>10^4$ compounds). Thus, it is advised to use a small number, typically *2* to *4*. Default: *1*.
- *ncpus_ML_algo* (integer). Number of cpus over which the ML training/test/validation is parallelized, for those ML algorithms which allow parallelization. Default: *1*.
- *ncpus_cv_traintest* (integer). Number of cpus over which the k-fold cross validation is parallelized. Should be an integer divisor of *n_test_train_splits*. Default: *1*.
- *ncpus_feat_importance* (integer). Number of cpus over which the evaluation of features importance is parallelized. Default: *1*.
- *ncpus_cv_regularization_combo* (integer). Number of cpus over which the cross validation of the L-normalized initial training of the feature combination is performed. This only applies to those algorithms that involve cross validation, i.e. those keywords in *regularized_model_featcombo* that end in "*cv*". Unless the number of cross-validation batches is manually changed inside the code, this variable should be set to 1 or 5. Default: *1*.

IPSE_CH module

This module evaluates (and plots) convex hulls for chemical systems. The convex hull (CH) is a mathematical construct adopted to establish whether a compound/material is thermodynamically stable (see, for example ref. 13). To build a CH for a given chemical system (i.e. set of elements), one needs an exhaustive list of compositions with their corresponding formation energy (assumed to be the one of the lowest-energy polymorph, see IPSE_ML module section). Through IPSE_CH, the user can build a convex hull based on data that are: fetched from a mongoDB database, and/or read from a file, and/or calculated on-the-fly with a ML model (typically generated through the IPSE_ML module). The construction of the CH itself is done through the *phase_diagram* module[19] of the pymatgen library[20,21]. Powder X-ray diffraction (XRD) patterns can also be simulated, if the crystal structure for the desired entries in the phase diagram is provided.

How to run

Similarly to the IPSE_ML module, the input options are set directly in python format in the file *input_IPSE_CH.py* (ver. 1.0.1). The IPSE_CH module is thus simply run by launching python with the main file:
python IPSE_CH_main.py

Input Keywords description

General options:

- *CH_elements* (list of string). Chemical elements for which the CH is calculated (also referred to as 'chemical system'). Example: *CH_elements=['Bi', 'S', 'Br']*. Default: empty list (*i.e.* this option must be defined by the user).
- *data_types_CH* (list of strings). What data sources are used to build the CH (mongoDB, file, or ML, see module description above). Allowed options: *"mongo"*, *"ML"*, and/or *"file"*. Default: empty list.
- *print_CH* (boolean). Whether the CH data, namely the compound compositions and their distance from CH, are printed on a file. The file is named: *convex_hull_<elements>.out* , where <elements> are the chemical elements of the chemical system under investigation, defined through the keyword *CH_elements*. Default: *True*.
- *threshold_instability* (float). Distance from the CH, in eV/atom, below which compounds are consider metastable rather than unstable (this only affects how compounds are categorized in the *convex_hull_<elements>.out* file. Default: *0.05*.
- *print_unstable* (boolean). Whether unstable compounds, that is compounds that are above the CH more than the *threshold_instability* value, are printed in the *convex_hull_<elements>.out* file. This can be useful to avoid printing large files when a large number of materials is used to build the CH, *e.g.* when reading data from the MongoDB database. Default: *True*.
- *plot_CH* (boolean). If *True*, the CH for binary and ternary systems is plotted. Unless set otherwise by the user on their computer, the CH is plotted through the default web browser. Default: *False*.
- *save_CH* (boolean). If *True*, the CH is saved as an html file. Note: this file is for plotting purposes, the CH information is printed through the *print_CH* option. Default: *False*.

[19] https://pymatgen.org/pymatgen.analysis.html#pymatgen.analysis.phase_diagram.CompoundPhaseDiagram
[20] Ong, S. P., Wang, L., Kang, B., & Ceder, G. (2008). Li− Fe− P− O2 phase diagram from first principles calculations. *Chemistry of Materials*, *20*(5), 1798-1807.
[21] Ong, S. P., Jain, A., Hautier, G., Kang, B., & Ceder, G. (2010). Thermal stabilities of delithiated olivine MPO4 (M= Fe, Mn) cathodes investigated using first principles calculations. *Electrochemistry Communications*, *12*(3), 427-430.

- *input_totalE* (boolean). If *True*, the energies given in input are considered as total energies rather than formation energies. The file has to report the total formula (atoms in the simulation structure file) and the final total energy (not per atom). Moreover, the energies of pure elements have to be provided, either in the input files (if more than one is provided, the lowest value is retained) or through the specific keyword *elemental_energies*. Default: *False*.
- *elemental_energies* (dictionary). This keyword must be set if *input_totalE=True* and the elemental energies are not provided in the input file or MongoDB. In such a case, the dictionary must contain entries in the form <element name>:<total energy per atom>. Default: empty dictionary.
- *exclude_polymorphs* (boolean). If True, only the lowest-energy structure for each composition is considered when building the convex hull. Default: *True*.

ML options (to evaluate compounds stability on-the-fly with previously created ML model):

- *ML_model_file* (string). Name of the pickle file that stores the previously trained ML model. Default: *'ML_model.pkl'*.
- *ML_scaler_file* (string). Name of the pickle file that stores the features scaling parameters adopted when training the ML model (see above). Default: *'scaler.pkl'*.
- *compounds4ML* (list of strings). Here the user can insert a list of chemical formulas whose formation energy is evaluated though the ML model. Default: empty list (no specific compounds are evaluated).
- *max_elements_composition* (integer or list of integers). This keyword is adopted to scan the investigated chemical system (*i.e.* set of elements) through ML by evaluating the formation energy of stoichiometries throughout the compositional space. Specifically, this keyword specifies the maximum allowed number of atoms of each element in the generated stoichiometries. For example, if *CH_elements=["Fe","O"]* and *max_elements_composition=3* , Fe_3O, Fe_3O_2, FeO, FeO_2, Fe_2O_3, and FeO_3 compounds will be considered. If a single integer is provided, the same maximum is applied to all elements. If a list is provided, it has to be of the same length as *CH_elements*, and each value corresponds to the maximum count for a specific element. Default: *None* (i.e. no compounds are generated through this procedure).
- *use_features_subset* (boolean). This keyword is used when the ML model has been obtained by selecting a subset of the feature space (typically selecting the most important features). If True, only the features listed in the *features_subset_file* (see below) are adopted to evaluate the formation energy through the ML model. Default: *False*.
- *features_subset_file* (string). Name of the file from where the subset of features to be used in the ML model (see *use_features_subset*) are read. Default: *'impo_feats.txt'*.
- *include_ML_in_CH* (Boolean). If *False*, the compounds selected through the keywords *compounds4ML* and *max_elements_composition* are not considered to build the CH. That is, their distance from the CH formed by all compounds read from files or MongoDB is evaluated. Default: *False*.
- *charge_neutral_only* (Boolean). If *True*, only compounds that fulfil the charge neutrality condition as defined by the ML features (Appendix A) are considered, the remaining compounds are not even evaluated. As to ver. 1.0.1, charge-neutral compounds are those having the following feature all equal to zero: 'min_form_charge','min_form_charge_neg','min_form_charge_pos','min_form_charge_truecatan'. Note that this filter can only be applied if the above-mentioned features were present in the adopted ML model. Note also that this filter only applies to the compounds generated through *max_elements_composition*, not to those listed in *compounds4ML*. Default: *False*.
- *ncpus_features* (int). Number of cpus over which the feature calculation is parallelized. This procedure does not scale very well with the number of cpus unless a large number of data is adopted. Thus, it is advised to use a small number, typically 2-4. Default: *1*.

External File options:

- *compounds_files* (list of strings). Name of the file(s) from which formulas and energies are read. It can be free text or csv format, in which case the .csv extension must be present. Default: *None*.
- *file_has_header* (boolean). Whether the file has a header as first line (skipped if *True*). Default: *True*.
- *column_compound* (integer). Which column in the file contains the compound formulas. Default: *0*.
- *column_E* (integer). Which column in the file contains the formation energies (or total energies if *input_totalE=True*). Default: *1*.
- *column_label* (integer or *None*). If set, a label is assigned to each data point (and printed in the output convex hull file) as read from the corresponding column of the file. Default: *None*.
- include_file_compounds_in_CH (boolean). If *False*, the compounds read from file are not used to build the CH. Only their distance from the CH is calculated. Default: *True*.

MongoDB options:

- *db_server_name* (string). Name (address) of the MongoDB server. Default: *"mongodb://localhost:27017/"* (which is the address typically assigned to the MongoDB installed to the user's own computer).
- *db_name* (string). Name of the MongoDB database from which data are read. Default: *None*.
- *db_collection_name* (string). Name of the collection within the *db_name* database from which data are read. Default: *None*.

Additional Options:

- *calculate_XRD* (boolean). Whether the XRD pattern for selected compounds (see below) should be calculated. Note that, as of ver. 1.0.1, XRD patterns are usually calculated for compounds associated to the "file option" (see above). However, the option *structure_files_XRD* (see below) allows to calculate the pattern for any structure. Default: *False*.
- *stability_threshold_XRD* (float). Calculate the XRD pattern for all the compounds whose distance from the convex hull falls below this value. Set *None* not to use this option. Default: *0.0*.
- *structures_path* (string). Absolute or relative path where the code looks for the structures (vasp's *CONTCAR* file as of ver. 1.0.1) to calculate the XRD pattern. The structures are assumed to be located at the path formed by *structures_path* and the label as read in the file at *column_label*. That is: *structures_path*/*label*/CONTCAR. Default: *".."*.
- *structure_files_XRD* (list of strings). Alternatively to *structures_path*, another way to select structures for which the XRD pattern is to be calculated is to provide, through this option, the paths where the structures are located. Not that these paths are added to *structures_path*, thus if the user wants to provide the full path here, *structures_path* must be set to an empty string. Note that this option can be used also to simulate the XRD pattern for any structure (in any format accepted by pymatgen), regardless of the convex hull. Default: *None*.
- *twotheta_range* (2-element tuple). Range of 2θ (diffraction angle) where the XRD pattern is calculated, in degrees. Default: *(0,90)*.

Acknowledgements

The first version of this code was produced for the Project IEMAP (Italian Energy Materials Acceleration Platform) within the Italian Research Program ENEA-MASE (Ministero dell'Ambiente e della Sicurezza Energetica) 2021–2024 "Mission Innovation" (agreement 21A033302 GU no. 133/5-6-2021).

Appendix A. Semiconductor-specific feature representations

The ML approach adopted within the IPSE DISCO code rests on the idea of dividing elements in the compound between cations or anions, even if their actual oxidation state may differ from that labelling (see charge balance features in the *ID2* paragraph below). This distinction is adopted in the IPSE_ML module to create the dataset, as only compounds containing at least one element from the cations list and one element from the anions list are included in the dataset. Importantly, this anion-cation labelling is also adopted for calculating the features to represent each compound in the ML process (training, test, and validation/prediction). The feature representations specific of this code (that is, beside those from external libraries, see e.g. the *"matminer"* option in the *styles_ML_features* keyword) are explained below. They are categorized according to the labels given in *styles_ML_features* and *options_ML_styles*.

ID1: statistics of elemental properties

For a given chemical formula, its composing elements are divided into cations and anions, and for each of the two subgroups the minimum, maximum, and weighted average of the elemental properties listed in the *options_ML_styles* (ID1) are calculated and adopted as features.

ID2: charge balance, minimum sum of formal charges

For a given formula the possible charges of the composing elements are defined as the oxidation states defined in the Mendeleev library[22], excluding the zero oxidation state. The minimum (in absolute value) formal charge is then calculated for the compound and adopted as a feature. This can entail elements formally defined as anions acquiring a positive charge, and vice versa (e.g. S in Na_2SO_4). Other three features are added: the minimum formal charge considering only positive oxidation states for cations, the minimum formal charge considering only negative oxidation states for anions, and the minimum formal charge considering both only positive cations and only negative anions.

ID3: distribution of elements in the periodic table

The following quantities are adopted as features, here divided by types corresponding to the *options_ML_styles* keyword.

- Basic ID3 features, calculated when *'ID3'* is in *styles_ML_features* , regardless of *options_ML_styles*:
  - Anion-to-cation ratio (#anions / #cations)
  - Number of elements in the formula that are cations
  - Number of elements in the formula that are anions
- *'periods'* features:
  - 4 features: fraction of anions belonging to period 2, 3, 4, and 5 of the periodic table
  - 5 features: fraction of cations belonging to period 2, 3, 4, 5, and 6 of the periodic table
- *'groups'* features:
  - 3 features: fraction of anions belonging to group VA, VIA, and VIIA of the periodic table
  - Fraction of cations that are transition metals
  - Fraction of cations that are alkaline or alkaline earth metals (groups IA and IIA)
  - Fraction of cations that belong to groups IIIA, IVA, or VA of the periodic table
  - Number of elements in the formula that are transition metals
  - Number of elements in the formula that are alkaline or alkaline earth metals
  - Number of elements in the formula that belong to groups IIIA, IVA, or VA of the periodic table
- *'entropy'* features:
  - Shannon entropy of cations, defined as $S_{cat} = \sum_i^{M\ cations} \frac{n_i}{N} \ln\left(\frac{n_i}{N}\right)$, where n_i is the number of cation atoms of type i and N is the total number of atoms in the formula

[22] https://mendeleev.readthedocs.io/en/stable/data.html#oxidation-states

- Shannon entropy of anions, defined as $S_{an} = \sum_i^{M\ anions} \frac{n_i}{N} \ln\left(\frac{n_i}{N}\right)$, where n_i is the number of anion atoms of type i and N is the total number of atoms in the formula

ID4: statistics of atomic orbital properties:

The atomic orbital radii and energies are considered for this set of features. As of ver. 1.0.1, radii are taken from Waber & Cromer[23] and energies from Thakkar's group[24,25]. As for ID1, elements in the formula are divided into cations and anions, according to the corresponding IPSE_ML input options. Then the minimum, maximum, and stoichiometry-weighted average of the following quantities are calculated and adopted as features:

- 12 features: energy and radii of the s and p orbitals of anions (min, max, and weighted average for each)
- 6 features: energy and radii of the s orbitals of cations (min, max, and weighted average for each)

Appendix B. Density estimation procedure.

The density is estimated empirically through the following formula:

$$\rho = \frac{\sum_i^{N\ species} n * w_i}{V^{estimated}}$$

Where w_i is the atomic weight of the i-th element (species) in the formula, n its stoichiometric coefficient, and:

$$V^{estimated} = 1.428 * \frac{4}{3} * \pi * \sum_i^{N\ species} n * r_i^3$$

With r being the ionic radii, averaged over the possible oxidation states (excluding 0 and positive charges for anions and negative charges for cations). Note that because of this averaging procedure, the densities obtained are just a rough estimation, meant to exclude the strong outliers. The pre-factor 1.428 was arbitrarily added to give a better agreement with the data in the Materials Project library.

Appendix C. Setting up MongoDB database (local or AWS cloud).

MongoDB on the cloud through Amazon Web Services (AWS). Follow these steps:

- First, create an account on MongoDB Atlas and link it to an AWS billing account, following the steps illustrated on this webpage (if you don't have an AWS billing account, you need to create that, too, see aws.amazon.com).
- Once the account has been created, one needs to create a project (Fig. C1a), and, within that, a Cluster (Fig.C1b), selecting one of the available payment plans (a free 'sandbox' plan should also be available, see Fig. C1c). Note that these clusters are suspended after a period of inactivity, and they can be resumed on the website.
- [This step is already taken care of by the IPSE_DISCO code: to connect through pymongo, follow the steps at this link; another useful resource is this video]
- To obtain the link to the online database (to be given to IPSE_DB under the keyword db_server_name), in the Cluster page, click "connect" and then select "drivers". Under "1. Select your driver and version" , select Python (any recent version is fine). The string to be given as

[23] Waber, J. T., & Cromer, D. T. (1965). Orbital radii of atoms and ions. *Journal of Chemical Physics*, *42*(12), 4116-4123.

[24] Koga, T., Tatewaki, H., & Thakkar, A. J. (1993). Roothaan-Hartree-Fock wave functions for atoms with Z≤ 54. *Physical Review A*, *47*(5), 4510.

[25] Koga, T., Kanayama, K., Watanabe, T., Imai, T., & Thakkar, A. J. (2000). Analytical Hartree–Fock wave functions for the atoms Cs to Lr. *Theoretical Chemistry Accounts*, *104*(5), 411-413.

db_server_name is reported at step 3; typically it starts with “mongodb+srv://” and ends with “appName=<cluster_name>” (obviously, in this string, one needs to replace <db_password> with the actual password set on MongoDB Atlas)

Local MongoDB on one’s own computer with Ubuntu.

This website reports the step-by-step procedure to install MongoDB on your computer with Ubuntu OS. For Windows, follow these steps instead (but the author of this document only tested the Ubuntu version). The following list of commands were tested to work by the author of this document:

```
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc |    sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg    --dearmor
lsb_release -a
vi /etc/apt/sources.list.d/mongodb-org-7.0.list
vi mongodb-org-7.0.list
sudo mv mongodb-org-7.0.list /etc/apt/sources.list.d/
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ]
https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/7.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt-get update
sudo apt-get install -y mongodb-org
vi /etc/mongod.conf
ps --no-headers -o comm 1
```

Then, every time the local MongoDB needs to be used (to add or retrieve data), one has to run:

```
sudo systemctl start mongod [--port <port_number>]
```

If one wants to use IPSE_DB (or IPSE_ML) with a local MongoDB database, the address to give to db_server_name is mongodb://localhost:27017/ . 27017 is the default port number: to use another port, one needs to start mongo on another port, too, by adding “--port <port_number>” when starting Mongo.

Other useful commands are reported below.

To check whether Mongo’s running:

```
sudo systemctl status mongod
```

To open Mongo shell (and use commands further below):

```
mongosh
```

Within mongosh, to, respectively, show the available databases, ‘enter’ into one of them, delete the database, print the collections within the database one has entered, delete a collection from the database one has previously entered:

```
show dbs, use <db name>, db.dropDatabase(),db.getCollectionNames(),db.<collection name>.drop()
```

To delete, in the *chalco1* collection (just an example of a collection name), all entries based that fulfil a condition, in this case that the “source database” feature has value “aflow” (one has to previously enter the database where the chalco1 collection is stored, see above):

```
db.chalco1.deleteMany({"source_database":"aflow"})
```

To list all entries in collection *chalco1_queries* (just an example of a collection):

```
db.chalco1_queries.find()
```

To find all entries without "warnings" in the collection "semicond1" of database "allsemicond", after having done "use allsemicond" :

```
db.semicond1.find({"warnings": {"$exists":false}})
```

delete specific entries:

```
db.chalco1_queries.deleteOne({"_id": ObjectId("65d5fe717b3ac28ae07051c0")})
```

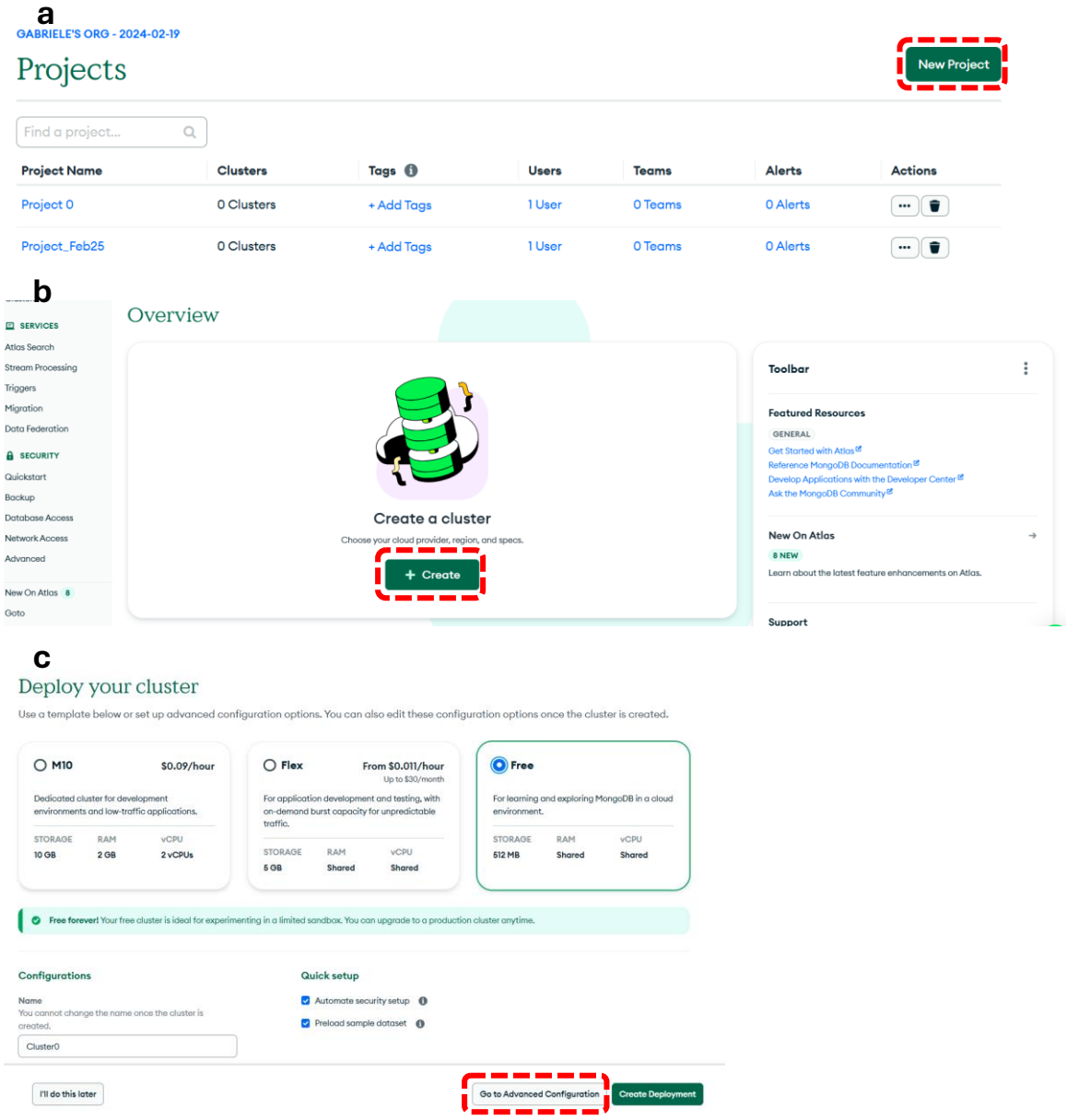


Figure C1 Instructions for the MongoDB cloud website. a) How to create a new project; b) how to create a new cluster; c) if, when creating a new cluster, the "free" plan (or other plans you desire) doesn't show up automatically, click on "go to advanced configurations"